SWT 22022 – Practical for Internet Application Development

Department of Information and Communication Technology

Faculty of Technology

Lab sheet :21
Reg. Number: SEU/IS/20/ICT/084
Academic Year :2020/2021
Date: 2024.11.06
Practical No :22

Title:

- E – Commerce App (Laravel) - II

Aims:

• Implement Laravel backend application

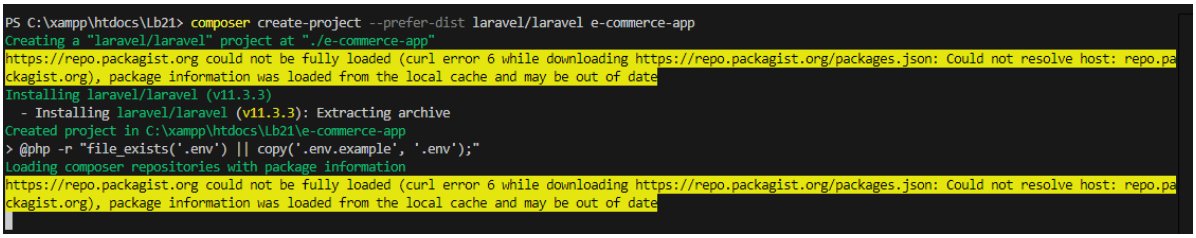
Tasks:

• Create a New Project
• Create model, migrations controller and request
• Implement CRUD operation
• Test with postman

Activities:

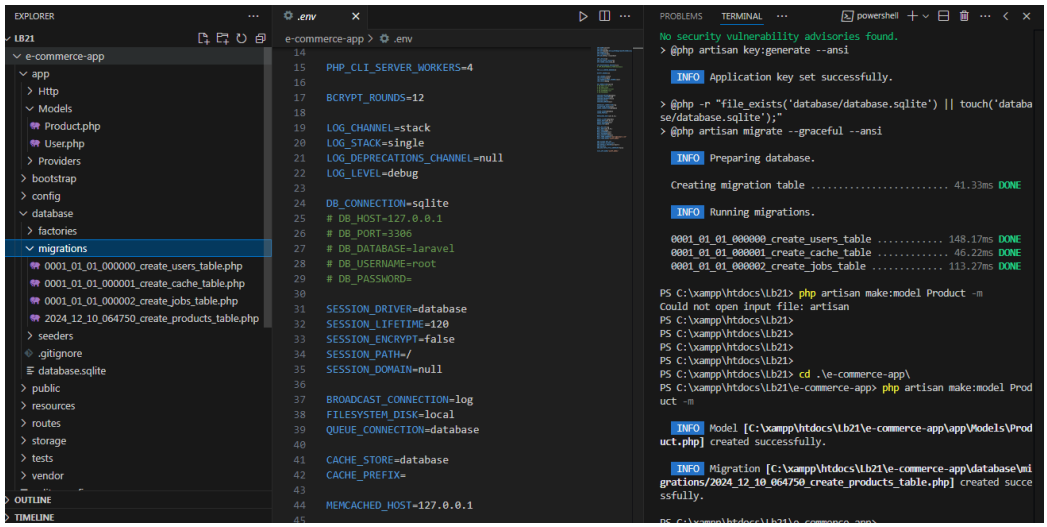
1. Create new project named “e-commerce-app”

   composer create-project --prefer-dist laravel/laravel e-commerce-app



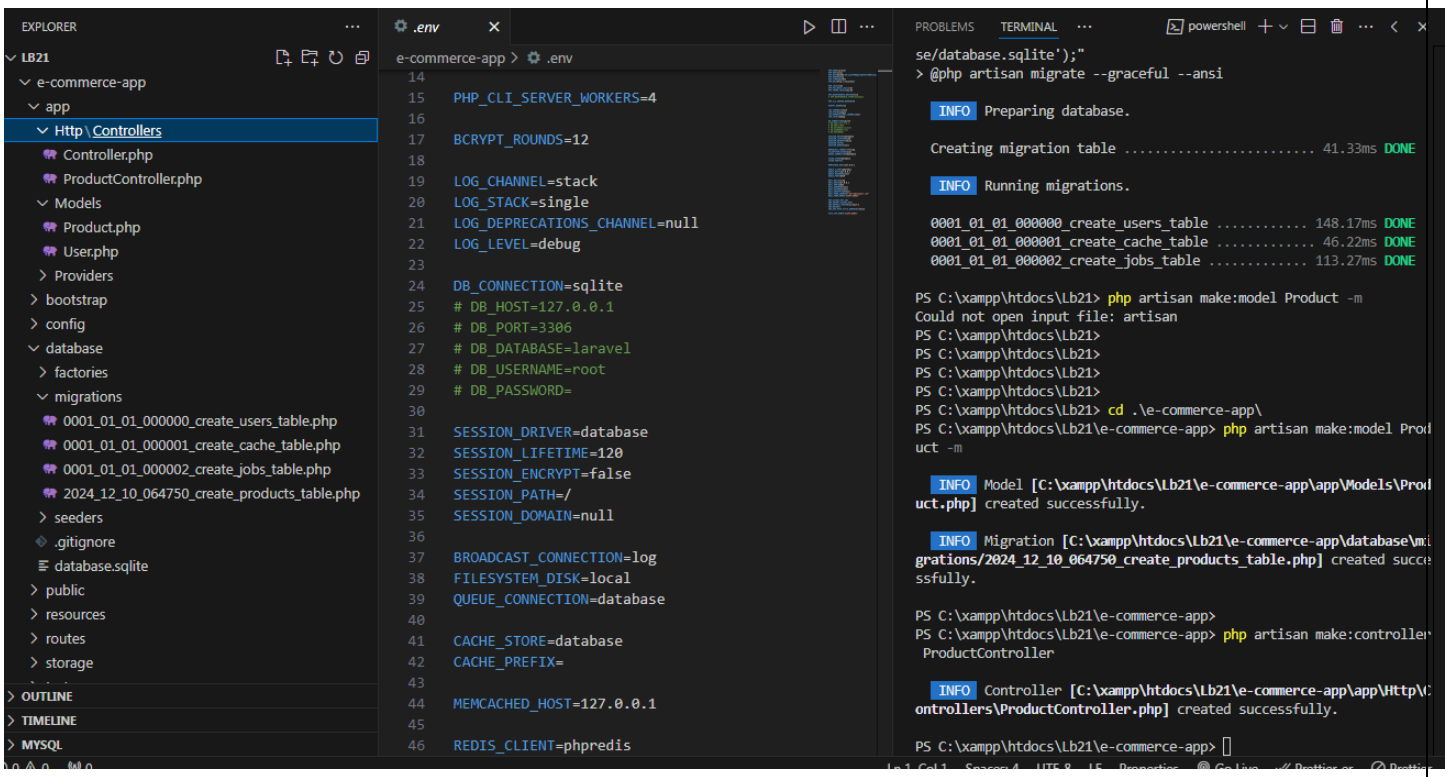
2. Create model & migration

   php artisan make:model Product -m



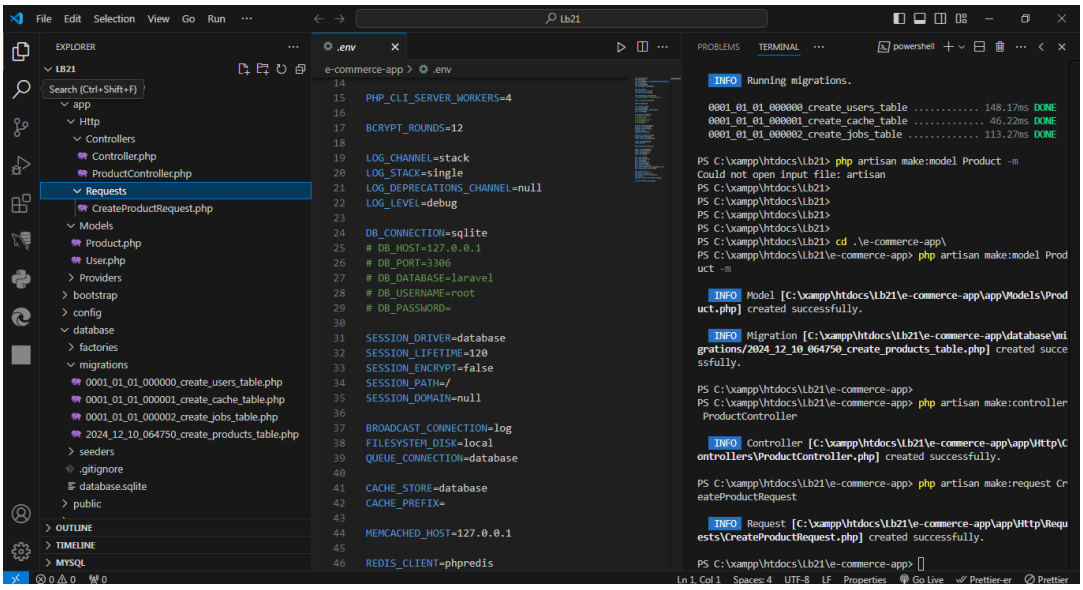
3. Create controller

php artisan make:controller ProductController

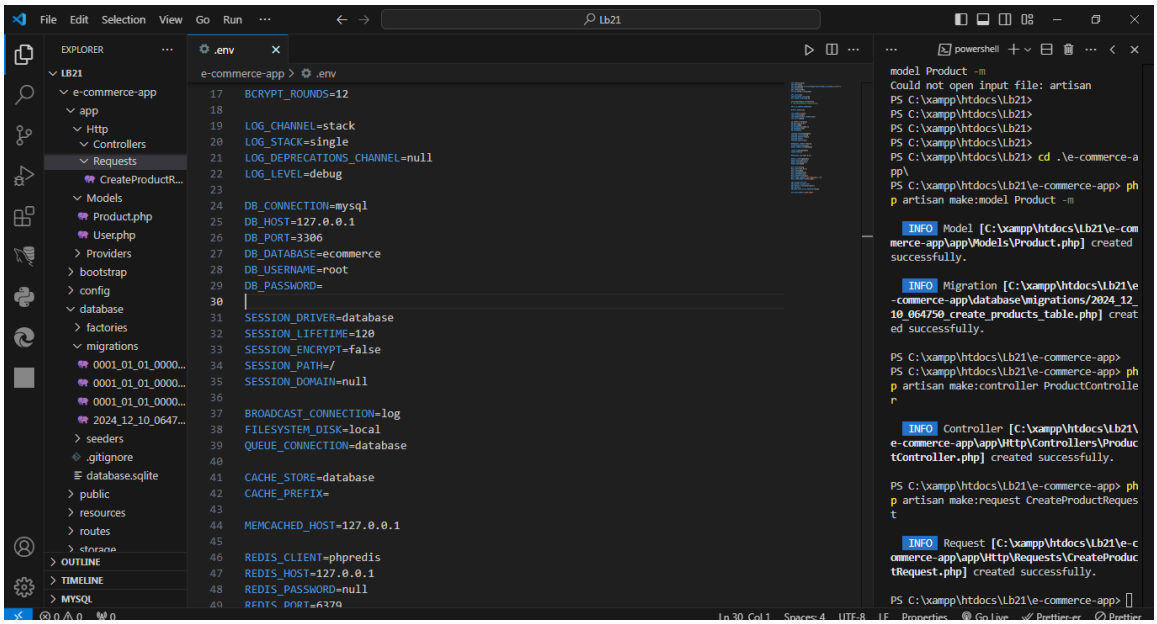


4. Create request

php artisan make:request CreateProductRequest



5. Update .env file



6. Complete Product model

```
class Product extends Model{
        use HasFactory;
        protected $fillable = [
                'name',
                'description',
                'image',
        'price',
        ];
}
```

Product.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
```

```
{
    use HasFactory;

    protected $fillable = [
        'name',
        'description',
        'image',
        'price',
    ];
}
```

7.Complete migrations

```
public function up(): void{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->text('description');
        $table->string('image');
        $table->decimal('price', 10, 2);
        $table->timestamps();
    });
}
```

2024_12_10_064750_create_products_table.php

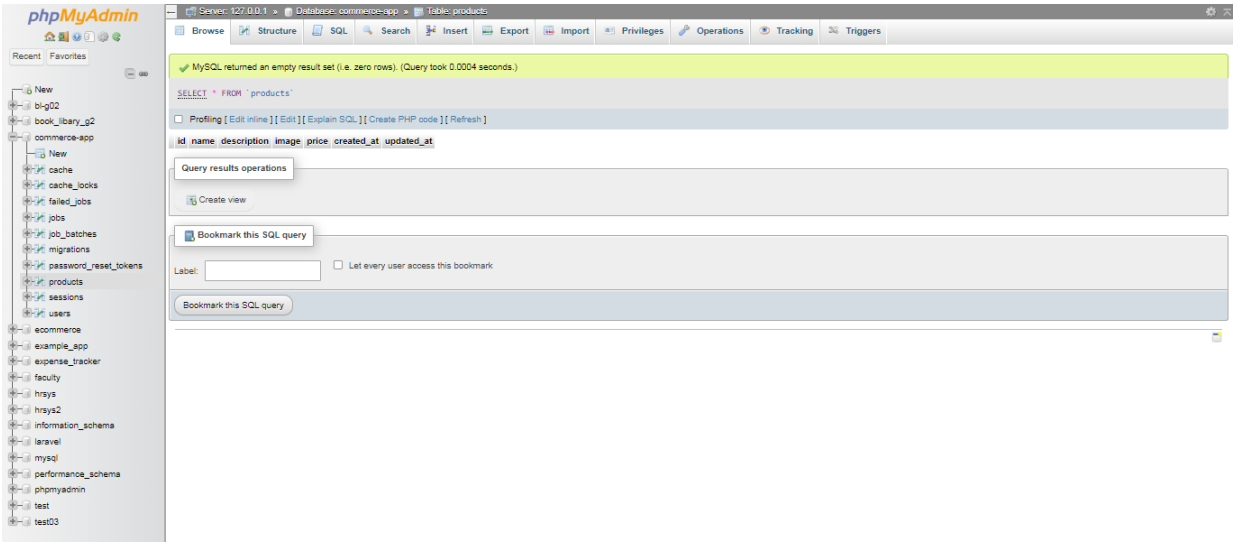
```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void{
        Schema::create('products', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->text('description');
            $table->string('image');
            $table->decimal('price', 10, 2);
            $table->timestamps();
        });
    }
```

```
    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('products');
    }
};
```

8. Run the migrations

php artisan migrate

```
PS C:\xampp\htdocs\Lb21\e-commerce-app> php artisan migrate

  WARN The database 'ecommerce' does not exist on the 'mysql' conn
ection.

  Would you like to create it? (yes/no) [yes]
❯

  INFO Preparing database.

  Creating migration table ..................... 100.62ms DONE

  INFO Running migrations.

  0001_01_01_000000_create_users_table ..... 516.84ms DONE
  0001_01_01_000001_create_cache_table ..... 92.26ms DONE
  0001_01_01_000002_create_jobs_table ..... 276.69ms DONE
  2024_12_10_064750_create_products_table ..... 51.95ms DONE

PS C:\xampp\htdocs\Lb21\e-commerce-app>
```



9. Implement CRUD operation

```
 public function all(){
    return Product::all();
}
public function store(CreateProductRequest $request){
    $imagePath = $request->file('image')->store('products','public');
    $product = Product::create(['name' => $request-
>validated('name'),'description' => $request->validated('description'),'price'
=> $request->validated('price'),'image' => $imagePath,]);
    return $product;
}
public function get($id){
    return Product::findOrFail($id);
}
public function update(Request $request, $id){
    $product = Product::findOrFail($id);
    return $product->update($request->all());
}
public function delete($id){
    $product = Product::findOrFail($id);
    return $product->delete();
}
```

```
<?php

use App\Models\Product;
use App\Http\Requests\CreateProductRequest;

class ProductController extends Controller
{
    // Get all products
    public function all()
    {
        return Product::all();
    }

    // Store a new product
    public function store(CreateProductRequest $request)
    {
        $imagePath = $request->file('image')->store('products', 'public');

        $product = Product::create([
            'name' => $request->validated('name'),
            'description' => $request->validated('description'),
            'price' => $request->validated('price'),
            'image' => $imagePath,
```

```
        ]);

        return $product;
    }

    // Get a single product by ID
    public function get($id)
    {
        return Product::findOrFail($id);
    }

    // Update an existing product
    public function update(Request $request, $id)
    {
        $product = Product::findOrFail($id);
        $product->update($request->all());

        return $product;
    }

    // Delete a product
    public function delete($id)
    {
        $product = Product::findOrFail($id);
        $product->delete();

        return response()->noContent();
    }
}
```

10. Complete request

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class CreateProductRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return false;
    }
```

```
    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string,
\Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
     */
    public function rules()
    {
        return [
            'name' => 'required|string|max:255',
            'description' => 'required|string',
            'price' => 'required|numeric|min:0',
            'image' => 'required|image|mimes:jpg,jpeg,png,gif|max:2048',
        ];
    }

}
```

11. Implement routes

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\ProductController;

Route::get('/', function () {
    return view('welcome');
});


Route::get('products', [ProductController::class, 'all']);
Route::post('products', [ProductController::class, 'store']);
Route::get('products/{id}', [ProductController::class, 'get']);
Route::put('products/{id}', [ProductController::class, 'update']);
Route::delete('products/{id}', [ProductController::class, 'delete']);
```

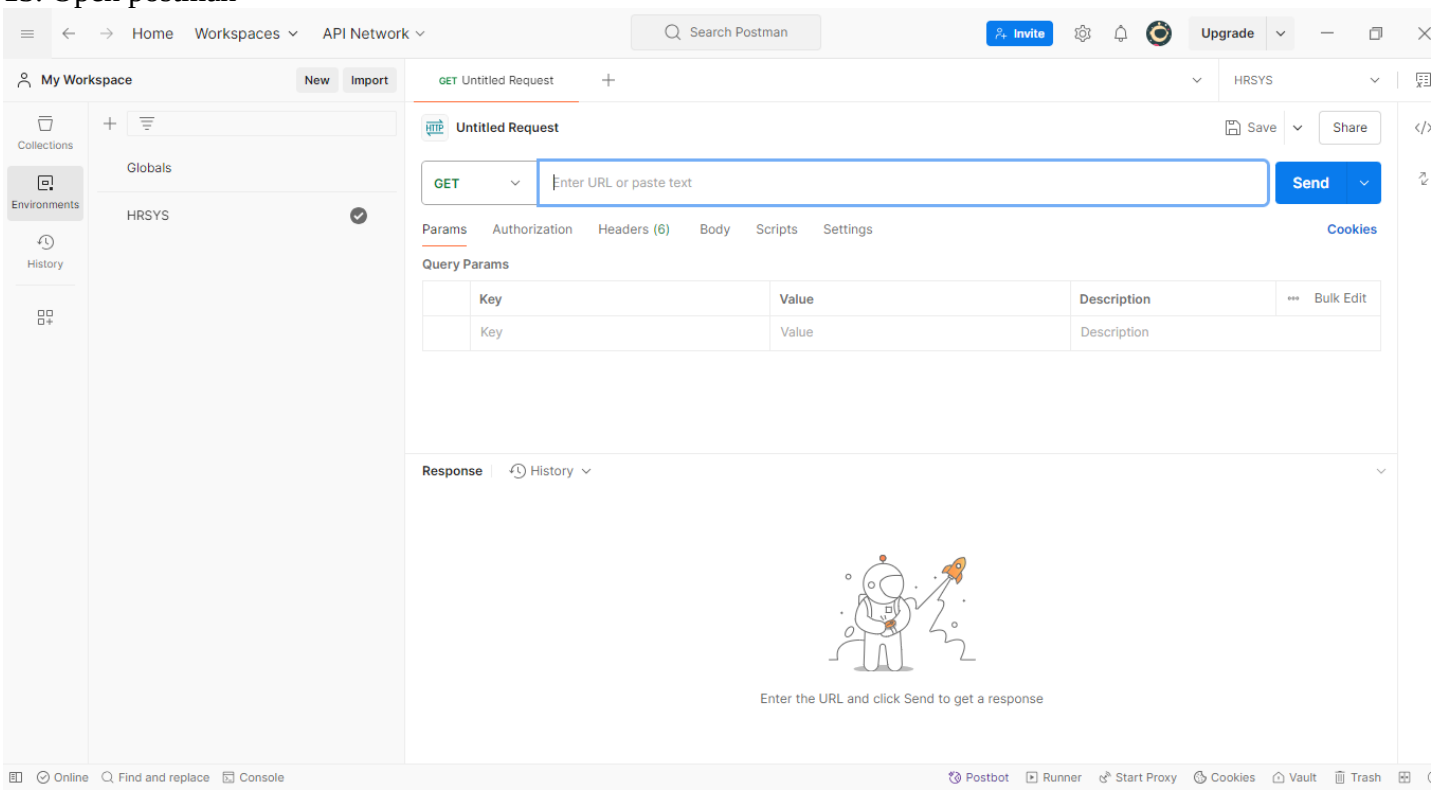
12. Implement new route for get token

```
Route::get('/token', function () {
      return csrf_token();
});
```

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\ProductController;
```

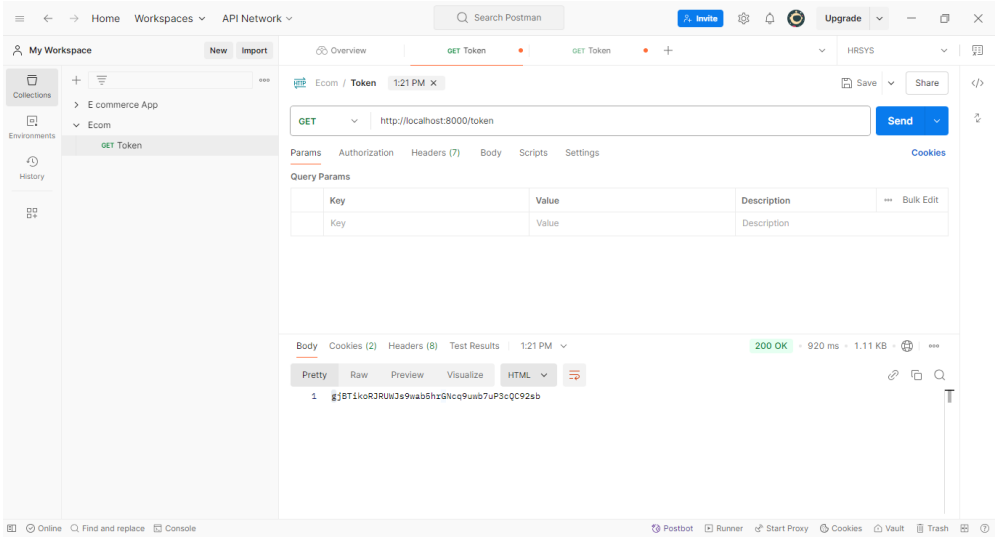
```
Route::get('/token', function () {
    return csrf_token();
});



Route::get('products', [ProductController::class, 'all']);
Route::post('products', [ProductController::class, 'store']);
Route::get('products/{id}', [ProductController::class, 'get']);
Route::put('products/{id}', [ProductController::class, 'update']);
Route::delete('products/{id}', [ProductController::class, 'delete']);
```

13. Open postman



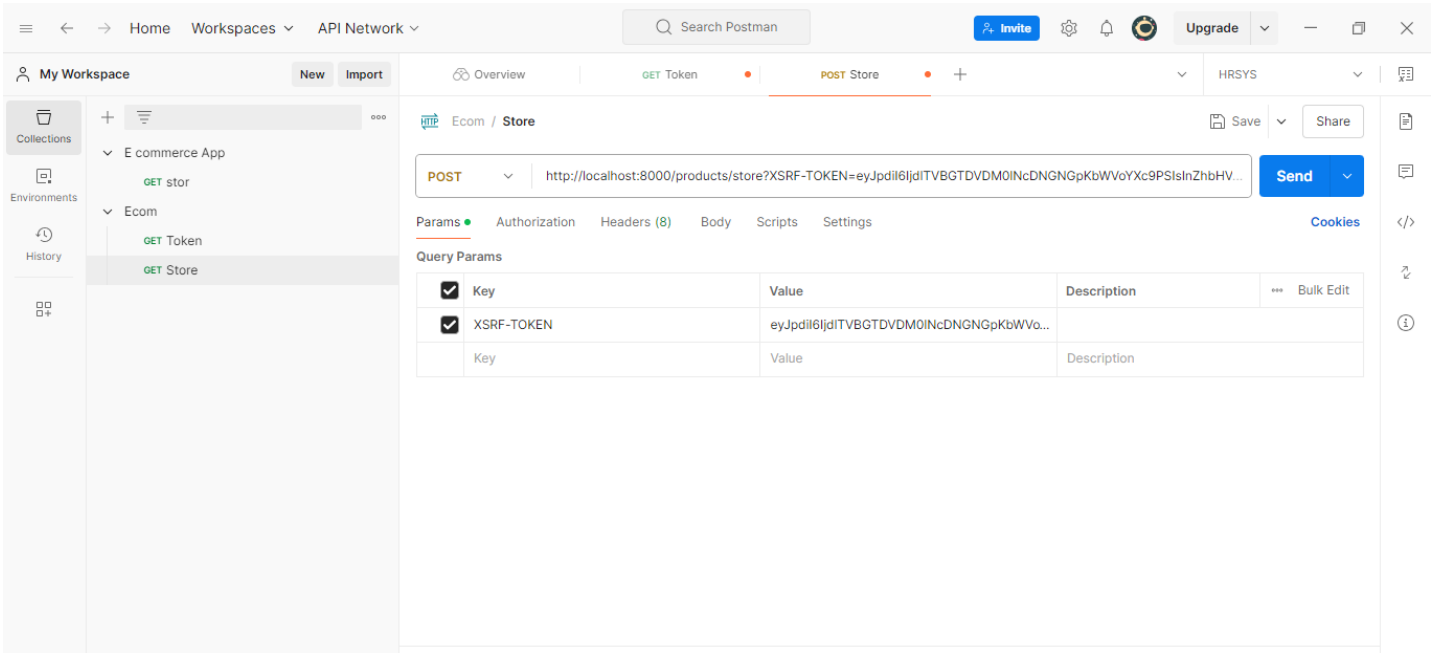
14. Run the token route

http://localhost:8000/token

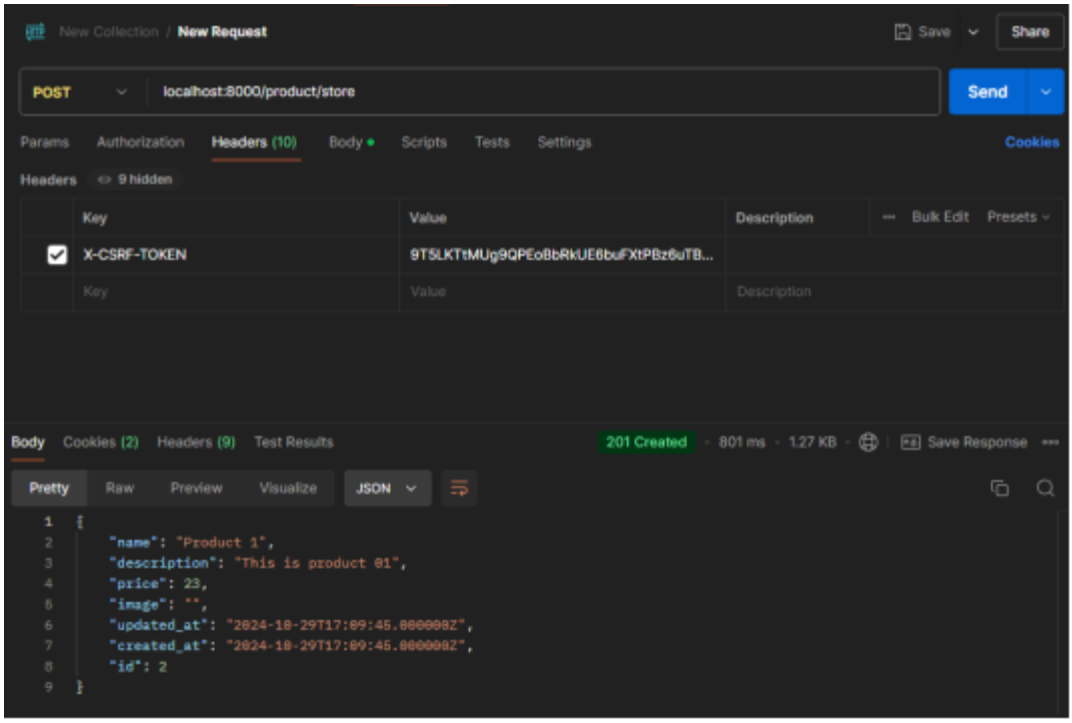


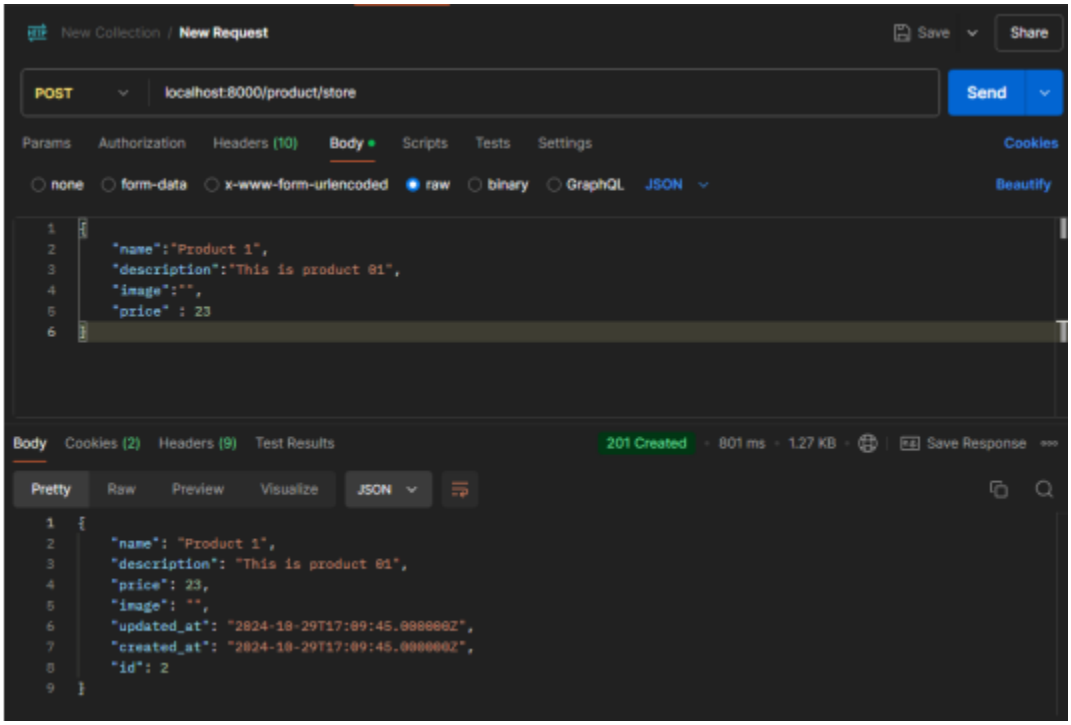
15. Save token in header

http://localhost:8000/products/store?XSRF-TOKEN=gjBTikoRJRUWJs9wab5hrGNcq9uwb7uP3cQC92sb



16. Run the requests


New Collection / New Request
Save
Share
POST
localhost:8000/product/store
Send
Params
Authorization
Headers (10)
Body
Scripts
Tests
Settings
Cookies
Headers
9 hidden
Key
Value
Description
Bulk Edit
Presets
X-CSRF-TOKEN
9T5LKTtMUg9QPEoBbRkUE6buFXtPBz6uTB...
Key
Value
Description
Body
Cookies (2)
Headers (9)
Test Results
201 Created
801 ms
1.27 KB
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"name": "Product 1",
"description": "This is product 01",
"price": 23,
"image": "",
"updated_at": "2024-10-29T17:09:45.000000Z",
"created_at": "2024-10-29T17:09:45.000000Z",
"id": 2
}


New Collection / New Request
Save
Share
POST
localhost:8000/product/store
Send
Params
Authorization
Headers (10)
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"name":"Product 1",
"description":"This is product 01",
"image":"",
"price" : 23
}
Body
Cookies (2)
Headers (9)
Test Results
201 Created
801 ms
1.27 KB
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"name": "Product 1",
"description": "This is product 01",
"price": 23,
"image": "",
"updated_at": "2024-10-29T17:09:45.000000Z",
"created_at": "2024-10-29T17:09:45.000000Z",
"id": 2
}

Discussion:

- **In this lab practical session, I worked on implementing the backend of an e-commerce application using Laravel. The first task was to create a new Laravel project, which provided the foundation for the application. I then created the necessary model, migration, controller, and request to manage products in the e-commerce system. This involved defining the product's attributes in the model, setting up the database schema through migrations, and handling validation using the request class. I implemented CRUD operations to allow for creating, reading, updating, and deleting products. Finally, I tested the endpoints using Postman to ensure the functionality was working correctly. This practical session helped me gain hands-on experience with Laravel's powerful features and how to build a simple backend API.**